

PRÁTICA 4: Blocos lógicos, ULA e Memória de dados

(Individual ou em dupla)

I. O QUE DEVE SER APRESENTADO (NA AULA DO DIA 18/09)

A **implementação da questão 1 deve ser feita durante a aula**. Durante a apresentação, os alunos deverão explicar as principais decisões tomadas durante a codificação, compilar o código, fazer a síntese e **demonstrar o funcionamento dos circuitos na placa FPGA DE-10 Altera** (use os displays de 7 segmentos para mostrar os valores dos operandos de entrada e os resultados em decimal).

II. O QUE DEVE SER ENTREGUE (TODAS AS QUESTÕES ATÉ DIA 24/09)

- Arquivo compactado nomeado “praticaX_aluno1_aluno2.zip”, onde X representa o número da prática entregue.
- **Código:** O arquivo entregue deve conter a pasta do projeto no Quartus II com todos os arquivos gerados, os arquivos de configuração das simulações em forma de onda no ModelSim e um relatório em PDF. Após o upload do arquivo, faça o download e certifique-se de que o arquivo será descompactado e que os arquivos são abertos corretamente.
- **Relatório:** O relatório deve incluir os seguintes componentes: capa, introdução com as suas palavras sobre o assunto da atividade prática, o projeto de todos os blocos incluindo a descrição de todas as decisões tomadas para implementação (faça uma ou mais figuras para ajudar na explicação do módulo e suas interconexões com outros blocos, se julgar necessário) e as simulações.
- **Simulações:** Acrescente ao relatório as imagens de formas de onda que prove o funcionamento correto de todos os circuitos descritos no Item III. Faça indicações em cada figura de forma de onda e escreva também um ou dois parágrafos explicando a sua simulação e o que você quis evidenciar sobre o funcionamento do circuito.

III. UM POUCO DE TEORIA

Unidade Lógica e Aritmética (ULA)

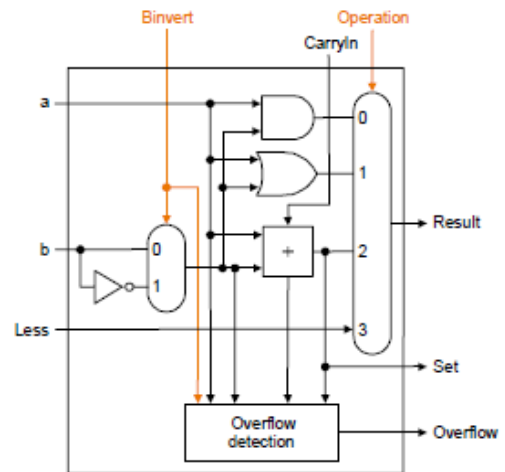
A Unidade Lógica e Aritmética (ULA) é fundamental para o processamento de dados feito pelo processador, representa os seus “músculos”. Trata-se de um bloco lógico responsável por executar as operações lógicas e aritméticas básicas que são necessárias para execução de uma instrução. Esse bloco funcional é constituído por duas entradas, um sinal de controle e uma saída. Na ULA, a execução de uma operação deve acontecer de forma sincronizada com os



demais blocos do processador, portanto a cada ciclo de clock uma operação deve ser executada.

A operação que deve ser executada com os dados de entrada (A e B) é determinada pelos sinais de controle (S) e o resultado é fornecido na saída (F). A complexidade da ULA é proporcional à complexidade do sistema em que será utilizada; assim, sistemas simples permitem o uso de ULAs simples e sistemas sofisticados exigem ULAs mais sofisticadas (com mais operações).

Nesta prática, construiremos a ULA do processador MIPS que deve ser capaz de executar cinco operações: duas lógicas: **and** e **or** e três aritméticas: **soma** (*add*), **subtração** (*sub*) e **comparação menor que** (*slt* - *set less than*). Lembre-se que a operação de subtração pode ser implementada através de uma soma com um operando em complemento de dois. A subtração consiste em somar o minuendo ao complemento de dois do subtraendo: $a - b = a + (\bar{b} + 1)$, como ilustra a Figura abaixo. Em complemento de dois, a soma de dois sinais de mesmo sinal gera um estouro se o resultado for de sinal diferente, neste caso a saída de *overflow* deve estar em nível lógico alto. Nesta prática, não iremos nos preocupar com as rotinas de tratamento de overflow.



Por último, a operação **Set on Less Than (slt)** é derivada da operação de subtração. Ela possui o seguinte formato: dado dois operandos (a) e (b), se (a) < (b), então o resultado é 1 indicando “verdadeiro”, caso contrário, o resultado é 0 indicando “falso”. Assim, todos os bits, com exceção do bit menos significativo, são fixados em zero. O bit menos significativo depende do resultado da comparação, que deve ser calculada a partir da subtração de (a) e (b) e tomando-se o bit de sinal: se $a - b < 0$, então $a > b$. Utiliza-se a própria função de subtração da ULA para obter este valor, modificando-se o último estágio. Inclui-se a entrada Less na ULA para prover a operação slt. A saída zero da ULA deve ter nível lógico alto, sempre que o resultado de uma operação for zero.

Memória de dados

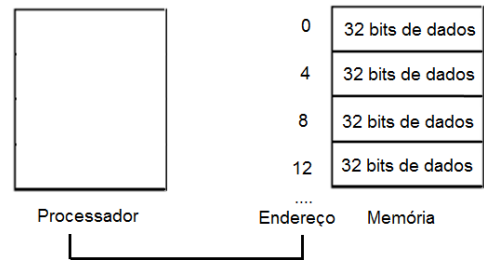
Nos computadores, as memórias são responsáveis pelo armazenamento de dados e instruções em forma de sinais digitais. Uma memória pode ser vista como um conjunto de elementos de armazenamento organizados como um array de N posições, onde cada posição é capaz de armazenar 1 byte (8 bits). Embora cada tipo de memória seja diferente em sua operação interna, certos princípios e conceitos são os mesmos para todos os sistemas de memória.

- Célula de memória: circuito capaz de armazenar um único bit.
- Palavra de memória: grupo de bits que representa uma unidade de informação (instruções ou dado de algum tipo), na arquitetura MIPS uma palavra contém 32 bits.
- Byte: grupo de 8 bits, unidade base de armazenamento em memória.
- Capacidade: expressa o número de bits que podem ser armazenados em uma memória.
- Endereço: identifica a posição de uma palavra na memória, cada palavra é armazenada em um único endereço. Nos sistemas digitais são representados em binário, embora números em hexadecimal sejam frequentemente usados, por conveniência, para representar o endereço.

Todo elemento de memória deve:

- Selecionar o endereço na memória que está sendo acessada por uma operação de leitura ou escrita;
- Selecionar uma operação de leitura ou de escrita para ser realizada;
- Fornecer os dados de entrada para serem armazenados na memória durante a operação de escrita;
- Manter os dados de saída vindos da memória durante uma operação de leitura;
- Habilitar (ou desabilitar) a memória de modo que ela responda (ou não) às entradas de endereço e ao comando de leitura e escrita.

Convencionalmente, o endereçamento nas memórias é a byte. Isso quer dizer que, cada posição de memória é capaz de armazenar apenas um byte. Para o MIPS, uma palavra possui 32 bits, ou seja, são necessários 4 bytes para armazenar uma palavra. Portanto, as palavras devem ser armazenadas em endereços que sejam múltiplos de 4. Esta necessidade é chamada restrição de alinhamento, presente em muitas outras arquiteturas.

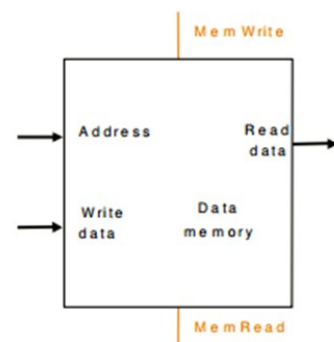


Ao invés de trabalhar com um único bloco de memória, por decisões de projeto da arquitetura, o MIPS utiliza dois blocos de memória similares, um dele é responsável por armazenar apenas as instruções e o outro armazena somente os dados. A memória de dados tem o seguinte funcionamento: Na borda de subida do ciclo de clock, se o sistema de leitura estiver ativo: dado um endereço, coloca na saída o dado armazenado. No caso de habilitação do sinal de escrita, dado um endereço e um dado na entrada, deve ser capaz de armazená-lo na posição especificada.

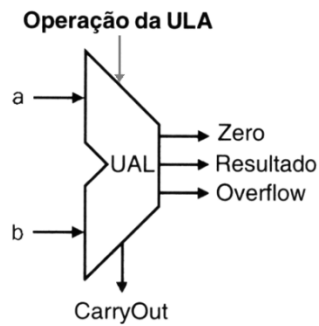
IV. O QUE DEVE SER FEITO

1. Faça uma pesquisa sobre as palavras reservadas do verilog "**generate**" e "**parameter**". Implemente o circuito de um **somador completo** com dois operandos de 16 bits. Utilize como bloco básico um somador completo com operandos de 1 bit (FA). Faça de forma que seja possível empregar o uso do *generate* e *parameter*. Use os displays de 7 segmentos para mostrar os valores dos operandos de entrada e os resultados em decimal. Caso não seja possível mapear todas as entradas e saídas para as chaves da Placa FPGA, você poderá usar apenas os 3 bits menos significativos de cada operando e do resultado.
2. Faça uma implementação **parametrizável** de uma memória de dados com capacidade de armazenamento de 64 bytes. Considere que as palavras têm 16 bits e que cada uma delas precisa de 2 bytes para ser armazenada completamente. Considere que a lógica de escrita é síncrona (na borda de subida do clock; se memWrite está habilitado) e a lógica de leitura é combinacional, isto é, assíncrona; se memRead está habilitado.

```
module    memoriaDados(address,    memWrite,
memRead, clock, writeData, readData);
    input [4:0] address;
    input memWrite, memRead, clock;
    input [15:0] writeData;
    output reg [15:0] readData;
    reg [7:0] memory [0:31];
endmodule
```



3. Faça uma implementação parametrizável de uma Unidade Lógica Aritmética, conforme descrita na Seção III.



Operação da ULA

000 = and
 001 = or
 010 = add
 110 = subtract
 111 = slt

```
module ULA32bits(a, b, ALUop, zero, result, overflow);
  input [31:0] a, b;
  input [2:0] ALUop;
  output [31:0] result;
  output overflow, zero;
endmodule
```

4. Implemente o circuito de um bloco **extensor de sinal**. Esse bloco deve receber na entrada um sinal de 16 bits e apresentar o mesmo valor, porém estendido para 32 bits.

Exemplo:

Entrada = 0000000000000001

Saída = 00000000000000000000000000000001